



JOIRIS (Solution)

Let N be the number of columns, and K the length of pieces. The remainder of division of N by K is denoted by $N \bmod K$. In the following, the columns are numbered in 0-indexed, i.e. the columns are numbered from 0 to $N - 1$. Let A_i be the number of blocks in the i -th column in the initial state of the game. Let B_i ($0 \leq i \leq K - 1$, $0 \leq B_i \leq K - 1$) be the sum of A_j 's modulo K for j with $0 \leq j \leq N - 1$ and $j \equiv i \pmod{K}$.

We claim that we can remove all blocks from the board if and only if both of the following conditions are satisfied:

- $B_0 = B_1 = \dots = B_{(N \bmod K)-1}$
- $B_{N \bmod K} = B_{(N \bmod K)+1} = \dots = B_{K-1}$

Here is a proof: by each of three possible operations (i.e. putting a piece horizontally/vertically, or deleting a row filled with blocks), the relative relations between $B_0, B_1, \dots, B_{(N \bmod K)-1}$ and $B_{N \bmod K}, B_{(N \bmod K)+1}, \dots, B_{K-1}$ do not change. (In other words, the equalities $B_0 = B_1 = \dots = B_{(N \bmod K)-1}$ are satisfied after the operation if and only if they are satisfied before the operation. The same is true for the equalities $B_{N \bmod K} = B_{(N \bmod K)+1} = \dots = B_{K-1}$.) So, if at least one of the above equalities is not satisfied, we cannot remove all blocks from the board.

On the other hand, we can construct an algorithm to remove all blocks from the board in the following way.

Step 1 For $i = 1, \dots, N - 1$, if $A_{i-1} > A_i$, put pieces vertically on the i -th column unless $A_{i-1} \leq A_i$. After this step, the board satisfies $A_0 \leq A_1 \leq \dots \leq A_{N-1}$, where A_i is the number of blocks in the i -th column.

Step 2 For $i = 0, \dots, A_{N-1}$, put pieces horizontally on the i -th row in a right-justified way as many as possible. After this step, the board satisfies $A_{K-1} = A_K = \dots = A_{N-1}$, i.e. the heights from the $(K - 1)$ -st column to the $(N - 1)$ -st column are the same.

Step 3 For $i = 0, \dots, K - 2$, put pieces vertically on the i -th column enough. After this step, the board satisfies $A_{K-1} = A_K = \dots = A_{N-1} = 0$, i.e. all blocks from the $(K - 1)$ -st column to the $(N - 1)$ -st column disappear.

Step 4 Since $B_0 = B_1 = \dots = B_{(N \bmod K)-1}$ and $B_{N \bmod K} = B_{(N \bmod K)+1} = \dots = B_{K-1}$ in the initial state of the game, we have $A_0 \equiv A_1 \equiv \dots \equiv A_{(N \bmod K)-1} \pmod{K}$ and $A_{N \bmod K} \equiv A_{(N \bmod K)+1} \equiv \dots \equiv A_{K-1} \pmod{K}$ after Step 3. Since $A_{K-1} = 0$ after Step 3, we must have $A_{N \bmod K} \equiv A_{(N \bmod K)+1} \equiv \dots \equiv A_{K-1} \equiv 0 \pmod{K}$. Putting pieces vertically, we can easily remove all blocks from the $(N \bmod K)$ -th column to the $(K - 2)$ -nd column.

Step 5 After Step 4, the board satisfies $A_0 \equiv A_1 \equiv \dots \equiv A_{(N \bmod K)-1} \pmod{K}$ and $A_{N \bmod K} = \dots = A_{N-1} = 0$. Now we can easily remove all blocks from the board by putting pieces vertically from the 1-st column to the $((N \bmod K) - 1)$ -st column, and putting pieces horizontally from the $(N \bmod K)$ -th column to the $(N - 1)$ -st column.

The proof of the claim is complete, and an algorithm is given.



Selling RNA Strands (Solution)

Subtask 1 [10 points]

$N, M, |S_i|, |P_j|, |Q_j|$ are small. We can solve this subtask by checking all RNA sequences for each order.

Subtask 2 [25 points]

Let's consider a simpler version of this problem:

There are N RNA sequences $S_1, \dots, S_N$ and M orders $P_1, \dots, P_M$. For each j ($1 \leq j \leq M$), compute the number of i such that the first $|P_j|$ characters of S_i are P_j .

We can store all of $S_1, \dots, S_N$ to a trie. Now we identify a string X and the node of the trie which corresponds to X . For any two strings X, Y , the following are equivalent:

- X is a prefix of Y , that is, the first $|X|$ characters of Y are X .
- Node X is an ancestor of node Y .

Let $C(S)$ be the number of occurrences of S in $S_1, S_2, \dots, S_N$. Then the problem is computing $\sum_{X: \text{predecessor of } P_j} C(X)$ for each j .

This can be done by precalculation with DFS, but here we use a different approach. Using the Euler-Tour technique, checking whether node x is an ancestor of node y can be reduced to checking whether a point (corresponding to y) is included in a range (corresponding to x). Now we get an algorithm for this problem:

- Store $S_1, \dots, S_N, P_1, \dots, P_M$ to a trie.
- Using the Euler-Tour technique, compute the range and the point for nodes in the trie. Note that we need not to compute the values for nodes which don't correspond to any of $S_1, \dots, S_N, P_1, \dots, P_M$.
- For each j , count the number of i such that the point corresponding to node S_i is included in the range corresponding to node P_j .

Let's go back to the original problem. Now we have the constraint by Q_j as well as by P_j . We saw that handling P_j can be done by a trie and the Euler-Tour technique. It can be easily checked that the same approach can be used for handling Q_j if we consider $S_1^R, \dots, S_N^R, Q_1^R, \dots, Q_M^R$ (where S^R is the reverse string of S).

Finally, we get an algorithm as follows:

- Store $S_1, \dots, S_N, P_1, \dots, P_M$ to a trie.



- Using the Euler-Tour technique, compute the range and the point for nodes in the trie.
- Do the same for $S_1^R, \dots, S_N^R, Q_1^R, \dots, Q_M^R$.
- For each j , compute the answer by checking two ranges.

Subtask 3 [25 points]

Let $\Sigma_S, \Sigma_P, \Sigma_Q$ be $|S_1| + \dots + |S_N|, |P_1| + \dots + |P_M|, |Q_1| + \dots + |Q_M|$, respectively.

We saw that this problem becomes relatively easy if we can ignore P or Q . Now consider the following algorithm:

- For each j , extract only RNA sequences whose last $|Q_j|$ characters are Q_j . Then compute the number of RNA sequences in the extracted ones whose first $|P_j|$ characters are P_j .

The later part of the algorithm can be done by using a trie. The former part also can be done with a trie. This algorithm is not so efficient because it may be possible that all of the RNA sequences are extracted and thus stored to a trie for each j . In this case, it works in $O(M\Sigma_S)$. But we can add a (seemingly small) optimization:

- Extract RNA sequences only once for the same Q .

In fact, it significantly reduces the complexity to $O(\Sigma_S \sqrt{\Sigma_Q} + \Sigma_P + \Sigma_Q)$.

Here is a proof. For each i , S_i is extracted only if Q_j is a suffix of S_i (that is, the last $|Q_j|$ characters of S_i are Q_j). Let L be the set of such Q_j 's. Each string of L has different length, so $\sum_{X \in L} |X| \geq 1 + \dots + \#L = \frac{(\#L + 1)\#L}{2}$.

Also $\sum_{X \in L} |X| \leq \Sigma_Q$ holds, so we have $\frac{(\#L + 1)\#L}{2} \leq \Sigma_Q$. Thus $\#L = O(\sqrt{\Sigma_Q})$. It means that each S_i is added to a trie $O(\sqrt{\Sigma_Q})$ times. Adding S_i to a trie once can be done in $O(|S_i|)$. So adding strings to tries can be completed overall in $O(\Sigma_S \sqrt{\Sigma_Q})$. Now we have tries, so computing the answer can be done by accessing node P_j of a trie. This can be done in $O(\Sigma_P)$. Also, checking all of $Q_1, \dots, Q_M$ itself takes $O(\Sigma_Q)$ time.

After all, it was proved that this algorithm works in $O(\Sigma_S \sqrt{\Sigma_Q} + \Sigma_P + \Sigma_Q)$.

Subtask 4 [40 points]

The approach to the solution for Subtask 2 is reducing the problem to the following:

There are N points X_i, Y_i and M rectangles $[A_j, B_j] \times [C_j, D_j]$ (whose axes are parallel to the X or Y axis) on a 2D plane. For each rectangle, compute the number of points which are included in it.

If we ignore irrelevant nodes of tries, the coordinates of the points and the rectangles will be integers between 0 and $N + M$.

Let $K(A, B, C, D)$ be the number of i such that $A \leq X_i \leq B$ and $C \leq Y_i \leq D$. As the coordinates of the points are not less than 0, we have $K(A, B, C, D) = K(A, B, -1, D) - K(A, B, -1, C - 1)$. So we just need to compute $K(A_k, B_k, -1, D_k)$ for some queries A_k, B_k, D_k efficiently. This can be done by the following algorithm:



- Sort the points and the queries by Y_i (for points) or D_k (for queries). If a point and a query have the same value, make the point appear earlier in the sequence.
- Initialize an array $V_0, \dots, V_{N+M}$ with 0.
- Scan the sorted sequence from the beginning and do the following:
 - For a point (X, Y) , set $V_X \leftarrow V_X + 1$.
 - For a query $(A, B, -1, D)$, answer $V_A + V_{A+1} + \dots + V_B$.

In this algorithm, we need to do array operations efficiently. This can be done with a Binary Indexed Tree.



Skyscraper (Solution)

Subtask 1 [5 points]

Brute force. Try every permutation. $O(N!)$

Subtask 2 [15 points]

Deciding each of the numbers from the leftmost position.

This can be solved by using bit DP. We calculate $dp[S][j][i]$, where S is the set of used number in the array, j is the last chosen number, and i is the current total sum of all differences of two adjacent numbers.

From each DP state, you can choose one of the unused number, so the total time complexity will be $O(2^N \cdot L \cdot N^2)$.

Subtask 3 [80 points]

In this Subtask, you should decide the relative positions of each number from the smallest value.

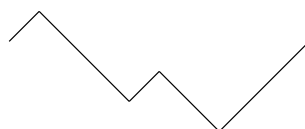


Figure 1

In other words, you should choose positions from below (see Figure 1).

This can be solved by DP. We consider $dp[i][j][k][l]$, where i means we calculate the number of ways to put the i -th smallest element, j is the number of connected components, k is the current total cost, and l is the number of elements already used for the leftmost or the rightmost edges. The treatment of connected components is tricky. When you add the i -th smallest element, either (1) it is disconnected with other connected components, (2) it is connected with one connected component only, or (3) it connects two connected components.

The additional cost on each phase depends on the value $A_i - A_{i-1}$ (after sorting $A_1, A_2, A_3, \dots, A_N$), and the number of connected components.

For further description about this DP, see “AppleTrees” from Member SRM 489¹, “Seatfriends” from SRM 625², or “SumOverPermutations” from SRM 666³.

¹https://community.topcoder.com/stat?c=problem_statement&pm=11213

²https://community.topcoder.com/stat?c=problem_statement&pm=12909

³https://community.topcoder.com/stat?c=problem_statement&pm=13764